PostgreSQL 인덱스 종류 학습 노트

날짜: 2026-05-22
복습 예정: D+1, D+3, D+7

한 줄 요약

책의 목차처럼, 데이터를 빠르게 찾기 위해 미리 만들어두는 지름길.

---

1. 비유

도서관에서 책 찾기:
- 인덱스 없음: 책 1만 권을 처음부터 끝까지 다 뒤짐 -> 느림
- 인덱스 있음: 목차 보고 바로 해당 페이지로 이동 -> 빠름

인덱스 종류 = 목차 방식이 다름. 상황에 맞는 방식 선택해야 최고 효율.

---

2. 인덱스 종류별 설명

B-tree (기본값)

```
[sql]
CREATE INDEX idx ON scan_executions (developer_id);
```

- 언제: 등호([=]), 범위([>], [<]) 검색
- 비유: 알파벳순 정렬된 목차
- Dev 2: [WHERE developer_id = 'dev-001']

복합 인덱스 (Composite)

```
[sql]
-- equality 컬럼 먼저, range 컬럼 나중 — 순서 중요!
CREATE INDEX idx ON scan_executions (developer_id, scanned_at DESC);
```

- 언제: 두 컬럼을 같이 검색할 때
- 비유: "저자명 + 출판연도" 두 가지 기준으로 정렬된 목차
- Dev 2: [WHERE developer_id = ? AND scanned_at > ?]

BRIN (Block Range Index)

```
[sql]
CREATE INDEX idx ON scan_executions USING brin (scanned_at);
```

- 언제: 시간순으로 쌓이는 데이터 (시계열)
- 비유: "1월 데이터는 앞쪽, 12월 데이터는 뒤쪽" 구역별 표시

- 장점: B-tree보다 크기 90% 작음
- Dev 2: [scanned_at >= date_trunc('month', now())]

GIN (Generalized Inverted Index)

```
[sql]
CREATE INDEX idx ON scan_issues USING gin (similar_issues);
```

- 언제: JSONB, 배열, 전문 검색
- 비유: 책의 색인(Index) — "SQL이라는 단어가 나오는 페이지 목록"
- Dev 2: [WHERE similar_issues @> '[{"pr_id": "98"}]']

부분 인덱스 (Partial Index)

```
[sql]
-- WHERE 조건에 해당하는 행만 인덱싱
CREATE INDEX idx ON scan_issues (scan_id, rule_id)
    WHERE severity = 'Critical';
```

- 언제: 전체 데이터 중 일부만 자주 조회할 때
- 비유: "베스트셀러만 따로 모아놓은 특별 목차"
- 장점: 인덱스 크기 대폭 감소 (Critical은 전체의 5~10%)
- Dev 2: Critical 에스컬레이션 조회 전용

커버링 인덱스 (Covering Index)

```
[sql]
CREATE INDEX idx ON scan_executions (developer_id)
    INCLUDE (id, scanned_at);
```

- 언제: SELECT에 필요한 컬럼을 인덱스에 포함시킬 때
- 비유: "목차에 페이지 번호뿐 아니라 챕터 요약도 포함"
- 장점: 테이블 본문 안 봐도 됨 -> 더 빠름
- Dev 2: [/insights/developer] 쿼리에서 테이블 스캔 제거

---

3. 요약 표

인덱스	언제 쓰나	Dev 2 적용
B-tree	등호/범위 검색	developer_id 검색
복합	컬럼 2개 이상 조건	developer + 날짜
BRIN	시계열 데이터	scanned_at
GIN	JSONB/배열 검색	similar_issues
부분	조건부 소수 데이터	Critical 이슈만
커버링	SELECT 컬럼 포함	insights 쿼리

복습 퀴즈

1. Q: BRIN 인덱스가 B-tree보다 좋은 상황은?
   A: (시간순으로 순차 증가하는 시계열 데이터. 크기가 90% 작음)

2. Q: 복합 인덱스에서 equality 컬럼을 먼저 써야 하는 이유는?
   A: (equality가 범위를 좁혀주므로 range 검색 효율이 높아짐)

3. Q: 부분 인덱스(Partial Index)를 쓰는 이유는?
   A: (자주 조회하는 일부 데이터만 인덱싱 -> 인덱스 크기 감소, 성능 향상)